```
0000 SEPARATE(¬IDL )¶--|-----------------------------------------------------------------------------------------------------------¶--|¬¬PROCEDURE¬STAT_GRMR¶--|-----------------------------------------------------------------------------------------------------------¶PROCEDURE¬STAT_GRMR¬( NOM_TEXTE :STRING¬) IS¶¶BEGIN¶  OPEN_IDL_TREE_FILE( NOM_TEXTE & ".lar" );¬¬¬¬¬¬¬--| COMMENCER PAR OUVRIR CELA¬POUR POUVOIR TRAVAILLER !¶¶  DECLARE¬¬¬¬¬¬¬¬¬¬--| PUIS DECLARER/INITIALISER¬CE DONT ON A BESOIN¶    USER_ROOT¬¬: TREE¬¬:= D( XD_USER_ROOT,¬TREE_ROOT);¶    GRAMMAR¬¬: TREE¬¬:= D( XD_GRAMMAR, USER_ROOT);¶    GR_RULE_S¬¬: SEQ_TYPE¬:= LIST( GRAMMAR);¶¶    STATE_SEQ¬¬: SEQ_TYPE¬:= (TREE_NIL,TREE_NIL);¬¬¬¬--| LISTE¬DE TOUS LES ETATS ENGENDRES¶    WORK_LIST¬¬: SEQ_TYPE¬:= STATE_SEQ;¬¬¬¬¬--| ETATS¬RESTANT ¬TRAITER¶    STATE_COUNT¬¬: INTEGER¬¬:= 0;¬¬¬¬¬¬¬--| NOMBRE D'ETATS¶    DUMMY_FOLLOW¬¬: TREE¬¬:= MAKE( DN_TERMINAL_S );¬¬¬¬--| LISTE¬DE SUIVI VIDE POUR PRNTSTAT¶¶    HASH_SIZE¬¬: CONSTANT INTEGER¬:= 2999;¬¬¬¬¬-- A PRIME¶    HASH¬¬¬: ARRAY (0 .. HASH_SIZE-1) OF¬TREE¬:= (OTHERS=> TREE_VOID);¶        -- DEFINED STATES¶¶        -- DATA FOR¬TABLES TO¬SPEED UP NEW STATE CALCULATION¶        -- FOR EACH¬SYMBOL, INDEX POINTS TO A CHAIN OF ITEMS WITH THE¶        -- GIVEN SYMBOL FOLLOWING THE POSITION MARKER (I.E., THOSE ITEMS¶        -- WHICH, AFTER SHIFTING, FORM THE CORE OF A NEW STATE).¶    TYPE INDEX_TYPE¬IS RECORD¬¬¬¬¬¬¬¬¬ ONE FOR EACH SYMBOL (- TER, + NONTER)¶¬¬  TIME¬¬: INTEGER¬:= 0;¬¬¬¬¬¬¬--| PASS (FROM STATE) AT WHICH USED¶
0010 ¬¬  F, L¬¬: INTEGER;¬¬¬¬¬¬¬--| PREMIER ET DERNIER ITEMS (N)¶¬¬END RECORD;¶    TYPE CHAIN_TYPE¬IS RECORD¶¬¬  N¬¬: INTEGER;¬¬¬¬¬¬¬--| ITEM SUIVANT (0¬POUR LA FIN DE CHANE)¶¬¬  T¬¬: TREE;¬¬¬¬¬¬¬¬--| UN ITEM¶¬¬  FIRST¬¬: BOOLEAN;¬¬¬¬¬¬¬--| PREMIER ITEM POUR LE SYMBOLE¶¬¬END RECORD;¶¶    INDEX¬¬: ARRAY( -INTEGER ( 170 ) .. 400 ) OF INDEX_TYPE;¶    CHAIN¬¬: ARRAY( 1 .. 100 )¬OF CHAIN_TYPE;¶    CHAIN_LAST¬: INTEGER;¶¶    --|-----------------------------------------------------------------------------------------------------------¶    --|¬¬PROCEDURE¬MAKE_ITEM¶    FUNCTION MAKE_ITEM ( ALTERNATIVE :TREE; SYLLABE_S :SEQ_TYPE; SYLLABE_NBR :INTEGER ) RETURN TREE¬IS¶      ITEM¬: TREE¬:= MAKE( DN_ITEM );¬¬¬¬¬¬¬¬--| FABRIQUER UN NOEUD ITEM¶    BEGIN¶      D¬( XD_ALTERNATIVE, ITEM, ALTERNATIVE );¬¬¬¬¬¬¬¬--| Y MENTIONNER L'ALTERNATIVE DONT IL VIENT¶      LIST( ITEM, SYLLABE_S );¬¬¬¬¬¬¬¬¬--| Y METTRE LA LISTE DES SYLLABES DE L'ALTERNATIVE¶      DI¬( XD_SYL_NBR, ITEM,¬SYLLABE_NBR );¬¬¬¬¬¬¬¬--| PORTER UN N DE¬SYLLABE INITIAL¶      D¬( XD_GOTO, ITEM, TREE_VOID );¬¬¬¬¬¬¬¬¬--| CHAMP¬INITIALISE  VIDE¶      D¬( XD_FOLLOW, ITEM, DUMMY_FOLLOW );¬¬¬¬¬¬¬¬--| CHAMP¬INITIALISE  LISTE VIDE¶      RETURN ITEM;¶    END;¶    --|-----------------------------------------------------------------------------------------------------------¶    --|¬¬PROCEDURE¬MAKE_STATE¶    FUNCTION MAKE_STATE RETURN TREE IS¶      STATE¬: TREE¬:= MAKE( DN_STATE );¶    BEGIN¶      STATE_COUNT := STATE_COUNT + 1;¬¬¬¬¬¬¬¬¬--| UN ETAT DE PLUS¶      DI(¬XD_STATE_NBR, STATE, STATE_COUNT );¬¬
0020 ¬¬¬¬¬--| METTRE LE N D'ETAT¶      STATE_SEQ := APPEND( STATE_SEQ, STATE );¬¬¬¬¬¬¬¬--| AJOUTER  LA LISTE D'ETATS¶¶      IF WORK_LIST.FIRST.TY /= DN_LIST THEN¬¬¬¬¬¬¬¬--| SI LA¬LISTE N'EST PAS INITIALISEE OU VIDE¶      IF WORK_LIST.FIRST.TY¬= DN_NIL THEN¬¬¬¬¬¬¬--| SI ELLE EST VIDE¶¬WORK_LIST.FIRST := STATE;¬¬¬¬¬¬¬¬¬--| POINTER L'ETAT CRE¶      ELSE¬¬¬¬¬¬¬¬¬--| LA LISTE DE TRAVAIL EST NON VIDE¶¬WORK_LIST.FIRST := STATE_SEQ.NEXT;¬¬¬¬¬¬¬--| LA LISTE DE TRAVAIL POINTE LE RESTE¬DE LA LISTE DES ETATS¶      END IF;¶      END¬IF;¶      RETURN STATE;¶    END;¶    --|-----------------------------------------------------------------------------------------------------------¶    --|¬¬PROCEDURE¬FORM_CLOSURE¶    PROCEDURE FORM_CLOSURE ( ITEM_SEQ :IN OUT SEQ_TYPE ) IS¶  INIT_NONTER_S¬¬: SEQ_TYPE¬:= (TREE_NIL,TREE_NIL);¬¬¬¬--| LISTE¬DES RGLES DE NON TERMINAUX QUI FORME LA FERMETURE¶    BEGIN¶¶FIND_RULES_FOR_CLOSURE:¬¬¬¬¬¬¬¬--| CHERCHER LES RGLES QUI VONT ALLER DANS LA FERMETURE DE¬L'ITEM¶      DECLARE¶      ITEM_S¬: SEQ_TYPE¬:= ITEM_SEQ;¶      ITEM¬: TREE;¶  BEGIN¶      WHILE NOT IS_EMPTY( ITEM_S ) LOOP¬¬¬¬¬¬¬¬--| TANT QU'IL Y A DES ITEMS¶¬POP( ITEM_S, ITEM );¬¬¬¬¬¬¬¬--| EN EXTRAIRE UN¶¬DECLARE¶¬  SYLLABE_S¬: SEQ_TYPE¬:= LIST( ITEM );¬¬¬¬¬¬--| PRENDRE SA LISTE DE SYLLABES¶¬  SYLLABE¬¬: TREE;¶¬BEGIN¶¬  IF NOT IS_EMPTY( SYLLABE_S ) THEN¬¬¬¬¬¬¬¬--| S'IL Y A DES SYLLABES¶¬    SYLLABE:= HEAD(¬SYLLABE_S¬);¬¬¬¬¬¬¬¬--| PRENDRE LA PREMIRE¶¬    IF SYLLABE.TY =¬DN_NONTERMINAL THEN¬¬¬¬¬¬¬--| SI C'EST UN NON¬TERMINAL¶¬
0030     DECLARE¶¬      RULE : TREE¬:= D( XD_RULE, SYLLABE );¬¬¬¬¬¬--| PRENDRE SA RGLE DE DEFINITION¶¬      BEGIN¶¬      IF RULE.TY /= DN_VOID¬THEN¬¬¬¬¬¬--| S'IL Y EN A UNE¶¬¬ INIT_NONTER_S := TERM_LIST.R_UNION( INIT_NONTER_S,¬¬¬¬-| MENTIONNER DANS¬LA LISTE DES NON TERMINAUX¶¬¬LIST( D( XD_INIT_NONTER_S, D(¬XD_RULEINFO, RULE )¬) )¶¬¬¬      );¶¬      END IF;¶¬      END;¶¬    END IF;¶¬  END IF;¶¬ END;¶      END LOOP;¶      END FIND_RULES_FOR_CLOSURE;¶¶INSERT_RULES_IN_CLOSURE:¬¬¬¬¬¬¬¬¬--| PORTER LES ALTERNATIVES DES RGLES TROUVEES DANS DES ITEMS CONSTITUANT LA FERMETURE¶      DECLARE¶      RULE¬¬: TREE;¶      BEGIN¶      WHILE NOT IS_EMPTY( INIT_NONTER_S ) LOOP¬¬¬¬¬¬¬¬--| TANT QU'IL Y A DES RGLES¶¬ POP( INIT_NONTER_S, RULE );¬¬¬¬¬¬¬--| EN EXTRAIRE UNE¶¬ DECLARE¶¬   ALTERNATIVE_S¬: SEQ_TYPE := LIST(¬RULE );¬¬¬¬¬¬¬--| PRENDRE LA LISTE DE SES ALTERNATIVES¶¬   ALTERNATIVE¬: TREE;¶¬ BEGIN¶¬   WHILE NOT IS_EMPTY( ALTERNATIVES_S ) LOOP¬¬¬¬¬¬¬--| TANT QU'IL Y A DES ALTERNATIVES¶¬     POP(¬ALTERNATIVE_S, ALTERNATIVE );¬¬¬¬¬¬¬¬--| EN EXTRAIRE UNE¶¬     ITEM_SEQ := APPEND( ITEM_SEQ,¬¬¬¬¬¬¬¬--| PEFIXER  LA LISTE D'ITEMS¶¬¬MAKE_ITEM( ALTERNATIVE, LIST(¬ALTERNATIVE ), 0 )¬¬¬¬--| UN NOUVEL ITEM POUR CETTE¬ALTERNATIVE¶¬¬¬);¶¬   END LOOP;¶¬ END;¶      END LOOP;¶      END INSERT_RULES_IN_CLOSURE;¶¶    END FORM_CLOSURE;¶    --|-----------------------------------------------------------------------------------------------------------¶    --|¬¬PROCEDURE¬MAKE_NEW_STATE¶    FUNCTION MAKE_NEW_STATE ( FROM
0040 _INDEX :INTEGER ) RETURN¬TREE IS¶      II¬¬¬: INTEGER¬¬:= INDEX(¬FROM_INDEX ).F;¶      STATE¬¬: TREE¬¬:= MAKE_STATE;¬¬¬¬¬¬--| FABRIQUER UN ETAT¶  NEW_ITEM_SEQ¬¬: SEQ_TYPE¬:= (TREE_NIL,TREE_NIL);¬¬¬¬--| LISTE¬D'ITEMS VIDE¶    BEGIN¶      WHILE II /= 0 LOOP¶      DECLARE¶¬ OLD_ITEM¬¬: TREE¬¬:= CHAIN(¬II ).T;¶¬ OLD_TAIL¬¬: SEQ_TYPE¬:= LIST( OLD_ITEM );¶¬ NEW_ITEM¬¬: TREE¬¬:= MAKE_ITEM(¶¬¬¬¬D( XD_ALTERNATIVE, OLD_ITEM ),¬¬¬¬--| POUR L'ALTERNATIVE DE L'ITEM PRE¶¬¬¬¬TAIL( OLD_TAIL ),¬¬¬¬¬¬¬--| AVEC LE RESTE DE LA LISTE¬DE SYLLABES¶¬¬¬¬DI( XD_SYL_NBR, OLD_ITEM) + 1¬¬¬¬¬--| UN N¬DE SYLLABE INCREMENTE DE 1¶¬¬¬¬);¶      BEGIN¶¬ NEW_ITEM_SEQ := APPEND( NEW_ITEM_SEQ, NEW_ITEM );¬¬¬¬¬--| PREFIXER L'ITEM¬ LA LISTE¶¬ II := CHAIN( II ).N;¬¬¬¬¬¬¬¬--| INDICE D'ITEM SUIVANT¶      END;¶      END LOOP;¶¶      FORM_CLOSURE( NEW_ITEM_SEQ );¬¬¬¬¬¬¬¬¬--| CALCULER LA FERMETURE DES¬ITEMS MIS¬EN LISTE¶    LIST( STATE,¬NEW_ITEM_SEQ );¬¬¬¬¬¬¬¬¬--| AJOUTER L'ETAT  LA LISTE¬D'ETATS¶      RETURN STATE;¶    END;¶    --|-----------------------------------------------------------------------------------------------------------¶    --|¬¬PROCEDURE¬MAKE_STATES¶    PROCEDURE MAKE_STATES IS¶      FROM_STATE¬¬: TREE;¶      FROM_NBR¬¬: INTEGER;¶      FROM_ITEM_SEQ¬¬: SEQ_TYPE;¶      FROM_ITEM¬¬: TREE;¶      FROM_INDEX¬¬: INTEGER;¶¶      --|-----------------------------------------------------------------------------------------------------------¶      --|¬¬PROCEDURE¬ITEM_INDEX¶      FUNCTION ITEM_INDEX ( IT :TREE ) RETURN INTEGER IS¶

0050       SYLLABE_S¬¬: SEQ_TYPE¬:= LIST( IT );¶      BEGIN¶      IF IS_EMPTY( SYLLABE_S ) THEN¶¬RETURN 0;¶      ELSE¶¬DECLARE¶¬  SYLLABE¬¬: TREE¬:= HEAD( SYLLABE_S );¶¬BEGIN¶¬  IF SYLLABE.TY = DN_TERMINAL¬THEN¬¬¬¬¬¬¬--| SYLLABE TERMINALE¶¬    RETURN - DI( XD_TER_NBR, SYLLABE );¬¬¬¬¬¬¬¬--| OPPOSE DU N DE¬TERMINAL¶¬  ELSE¬¬¬¬¬¬¬¬¬¬--| NON TERMINALE¶¬    DECLARE¶¬      RULE : TREE¬:= D( XD_RULE, SYLLABE );¬¬¬¬¬¬¬--| RGLE¬DE DEFINITION¶¬    BEGIN¶¬      IF RULE.TY = DN_VOID THEN¬¬¬¬¬¬¬¬--| PAS DE RGLE DE¬DEFINITION¶¬        RETURN 0;¬¬¬¬¬¬¬¬¬--| N 0¶¬      ELSE¬¬¬¬¬¬¬¬¬--| RGLE¬PRESENTE¶¬        RETURN DI( XD_RULE_NBR, D( XD_RULEINFO, RULE ) );¬¬¬¬¬--| LE N¬DE RGLE¶¬      END¬IF;¶¬    END;¶¬  END IF;¶¬END;¶        END IF;¶      END¬ITEM_INDEX;¶      --|-----------------------------------------------------------------------------------------------------------¶      --|¬¬PROCEDURE¬CALCULATE_GOTO¶      PROCEDURE CALCULATE_GOTO IS¶        TO_STATE¬¬: TREE¬¬:= TREE_VOID;¶        FROM_ALT¬¬: TREE¬¬:= D( XD_ALTERNATIVE, FROM_ITEM );¶        POSSIBLE_TO¬¬: TREE;¶        TEMP_ITEM¬¬: TREE;¶      HASH_CODE¬¬: INTEGER¬¬:= 0;¶        HASH_DELTA¬¬: INTEGER¬¬:= 1;¶        ELEMENT_COUNT¬: INTEGER¬¬:= 0;¶        II¬¬: INTEGER;¬¬¬¬¬¬¬¬ ITEM CHAIN NUMBER¶¶      --|-----------------------------------------------------------------------------------------------------------¶      --|¬PROCEDURE¬CHECK_POSSIBLE_TO¶      FUNCTION CHECK_POSSIBLE_TO ( FROM_INDEX :INTEGER; POSSIBLE_TO¬:TREE ) RETURN BOOLEAN IS¶¬II¬¬: INTEGER¬¬:= INDEX(¬FROM_INDEX ).F;¶¬NEW_ITEM_SEQ¬: SEQ_TYPE¬:= LIST (¬POSSIBLE_TO );¶¬OLD_ITEM¬¬: TREE;¶¬NEW_ITEM¬¬: TREE;¶      BEGIN¶¬LOOP¶¬  IF IS_EMPTY( NEW_ITEM_SEQ )¬¬¬¬¬¬¬--| PLUS D'ITEM¶¬    OR ELSE DI( XD_SYL_NBR, ( HEAD( NEW_ITEM_SEQ¬) ) ) = 0¬¬¬¬¬--| OU PLUS DE SYLLABE¶¬  THEN¶¬    RETURN II = 0;¶¬  END IF;¶¬  IF II =¬0 THEN¶¬    RETURN FALSE;¶¬  END IF;¶¬  OLD_ITEM := CHAIN( II ).T;¶¬  NEW_ITEM := HEAD ( NEW_ITEM_SEQ );¶¬  IF D( XD_ALTERNATIVE, OLD_ITEM ) /= D( XD_ALTERNATIVE, NEW_ITEM )¶¬    OR ELSE DI( XD_SYL_NBR,OLD_ITEM) +1 /= DI( XD_SYL_NBR,¬NEW_ITEM )¶¬  THEN¶¬    RETURN FALSE;¶¬  END IF;¶¬  II := CHAIN(¬ II ).N;¶¬  NEW_ITEM_SEQ := TAIL( NEW_ITEM_SEQ );¶¬END LOOP;¶      END CHECK_POSSIBLE_TO;¶¶    BEGIN¶      II := INDEX( FROM_INDEX ).F;¶      WHILE II /=¬0 LOOP¶¬ELEMENT_COUNT := ELEMENT_COUNT + 1;¶¬TEMP_ITEM¬:= CHAIN(¬II ).T;¶¬HASH_CODE¬:= ABS(¶¬¬¬HASH_CODE¶¬¬ - 28 * DI( XD_ALT_NBR, D( XD_ALTERNATIVE, TEMP_ITEM ) )¶¬¬ - 3  * DI( XD_SYL_NBR, TEMP_ITEM )¶¬¬ - 11 * ELEMENT_COUNT¶¬¬ );¶¬ II := CHAIN( II ).N;¶      END LOOP;¶      HASH_CODE := HASH_CODE MOD HASH_SIZE;¶¶      WHILE HASH(¬HASH_CODE¬) /= TREE_VOID LOOP¶¬POSSIBLE_TO := HASH( HASH_CODE );¶¬IF CHECK_POSSIBLE_TO( FROM_INDEX, POSSIBLE_TO ) THEN¶¬  TO_STATE := POSSIBLE_TO;¶¬  EXIT;¶¬END IF;¶¬IF HASH_DELTA >= HASH_SIZE THEN¶¬  PUT_LINE( "HASH TABLE OVERFLOW." );¶¬  RAISE PROGRAM_ERROR;¶¬END IF;¶¬HASH_CODE¬:= (HASH_CODE + HASH_DELTA) MOD HASH_SIZE;¶¬HASH_DELTA := HASH_DELTA + 2;¬¬¬¬¬¬¬-- SO HASH_CODE INCREASES BY N **
0070  2¶      END LOOP;¶¶      IF TO_STATE¬= TREE_VOID THEN¬¬¬-- DIDN'T¬FIND ONE¶¬TO_STATE := MAKE_NEW_STATE ( FROM_INDEX¬);¶¬HASH( HASH_CODE ) := TO_STATE;¶      END IF;¶¬¬    -- INSERT NEW STATE AS GOTO FOR ALL¬RULES WITH GIVEN NEXT SYMBOL¶      II := INDEX( FROM_INDEX ).F;¶      WHILE II /=¬0 LOOP¶¬FROM_ITEM¬:= CHAIN(¬II ).T;¶¬D( XD_GOTO, FROM_ITEM, TO_STATE );¶¬II := CHAIN( II ).N;¶      END LOOP;¶    END¬CALCULATE_GOTO;¶¶    BEGIN¶    WHILE NOT IS_EMPTY( WORK_LIST ) LOOP¶      POP( WORK_LIST, FROM_STATE );¶      FROM_NBR :=¬DI( XD_STATE_NBR, FROM_STATE );¶      IF STATE_COUNT MOD 20¬= 0 THEN¶¬PUT( '*');¶      END IF;¶¬¬    -- CONSTRUCT CHAIN STRUCTURE FOR THIS STATE¶      CHAIN_LAST := 0;¶      FROM_ITEM_SEQ := LIST( FROM_STATE );¶      WHILE NOT IS_EMPTY( FROM_ITEM_SEQ ) LOOP¶¬POP( FROM_ITEM_SEQ,¬FROM_ITEM¬);¶¶¬FROM_INDEX := ITEM_INDEX( FROM_ITEM );¶¬IF FROM_INDEX /= 0 THEN¶¬  CHAIN_LAST := CHAIN_LAST + 1;¶¬  DECLARE¶¬    CHAIN_I¬: CHAIN_TYPE RENAMES CHAIN(CHAIN_LAST);¶¬    INDEX_I¬: INDEX_TYPE RENAMES INDEX(FROM_INDEX);¶¬  BEGIN¶¬    IF INDEX_I.TIME¬/= FROM_NBR THEN¶¬      INDEX_I := (TIME=> FROM_NBR, F=> CHAIN_LAST, L=> CHAIN_LAST);¶¬      CHAIN_I := (N=> 0, T=> FROM_ITEM,¬FIRST=> TRUE);¶¬    ELSE¶¬      CHAIN( INDEX_I.L ).N :=¬CHAIN_LAST;¶¬      CHAIN_I := (N=> 0, T=> FROM_ITEM,¬FIRST=> FALSE);¶¬    END IF;¶¬    INDEX_I.L := CHAIN_LAST;¶¬  END;¶¬END IF;¶        END LOOP;¶¶        FOR CH IN 1¬.. CHAIN_LAST LOOP¶¬IF CHAIN(¬CH ).FIRST THEN¶¬  FROM_ITEM := CHAIN( CH ).T;¶¬  FROM_INDEX := ITEM_INDEX( FROM_ITEM );¶¬  CALCULATE_GOTO;¶¬END IF;¶        END LOOP;¶¶      END¬LOOP;¶    END MAKE_STATES;¶¶  BEGIN¶¶    DECLARE¶      RULE_S¬¬: SEQ_TYPE¬:= GR_RULE_S;¶      RULE¬¬: TREE¬¬:= HEAD( GR_RULE_S );¬¬¬¬--| PREMIRE RGLE¶      ALTERNATIVE_S¬¬: SEQ_TYPE¬:= LIST( RULE );¬¬¬¬¬--| SA LISTE D'ALTERNATIVES¶      ALTERNATIVE¬¬: TREE¬¬:= HEAD( ALTERNATIVE_S );¬¬¬¬--| LA PREMIRE ALTERNATIVE¶      SYLLABE_S¬¬: SEQ_TYPE¬:= LIST( ALTERNATIVE );¬¬¬¬--| SA LISTE DE SYLLABES¶      ITEM_SEQ¬¬: SEQ_TYPE¬:= APPEND( (TREE_NIL,TREE_NIL),¬¬¬--| METTRE EN LISTE¶¬¬¬¬¬¬    MAKE_ITEM( ALTERNATIVE, SYLLABE_S,¬0 )¬¬¬| LE PREMIER ITEM¶¬¬¬¬¬¬);¶    BEGIN¶      LIST( DUMMY_FOLLOW, (TREE_NIL,TREE_NIL) );¬¬¬¬¬¬¬¬--| METTRE UNE LISTE VIDE DANS LE SUIVI¶      FORM_CLOSURE(¬ITEM_SEQ );¬¬¬¬¬¬¬¬--| CALCULER LA FERMETURE DU PREMIER ITEM¶      LIST( MAKE_STATE, ITEM_SEQ );¬¬¬¬¬¬¬¬--| FABRIQUER UN PREMIER ETAT¬AVEC L'ITEM INTERESSANT L'ALTERNATIVE 1¬DE LA RGLE 1¶    END;¶¶    MAKE_STATES;¶¶    DECLARE¶      STATE_S¬: TREE¬:= MAKE( DN_STATE_S¬);¬¬¬¬¬¬¬--| FABRIQUER UN ACCS POUR LA LISTE D'ETAT¶    BEGIN¶      LIST( STATE_S, STATE_SEQ );¬¬¬¬¬¬¬¬¬--| Y PORTER LA LISTE CONSTITUEE¶    D( XD_STATELIST, USER_ROOT, STATE_S );¬¬¬¬¬¬¬--| METTRE DANS LA LISTE DE LA STRUCTURE DE DONNEES¶    END;¶  END;¶¶  CLOSE_IDL_TREE_FILE;¶END STAT_GRMR;¶
```